AIOPS-02: Anomaly Detection

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 2 of 8 | **Created:** May 2026 | **Last Updated:** 10/12/2026

Overview

Anomaly detection in Dynatrace is **Predictive AI in action** — the platform learns what normal looks like and surfaces deviations. Five mechanisms cover the practical detection landscape: static thresholds, auto-adaptive thresholds, seasonal baselines, multi-dimensional baselines, and novelty / forecasting.

Most teams over-rely on static thresholds and miss what adaptive and seasonal detection would catch. This notebook walks the five mechanisms, when to use each, and how to test them against your data using the Davis analyzer MCP tools.

Audience: Platform admin tuning detection; SRE writing custom alerts.

Outcome: A working understanding of which detector to pick for each metric type, and live examples against your tenant.

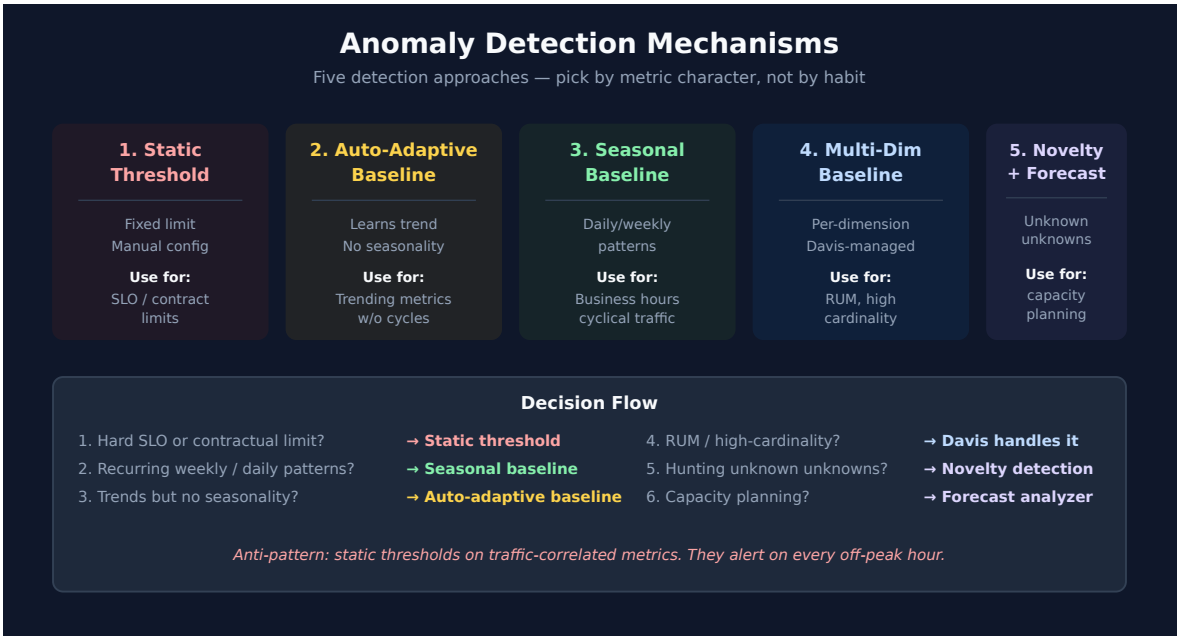


Table of Contents

1. The Five Detection Mechanisms
2. Picking the Right Detector
3. Configuration Surface: App vs. Settings vs. Code
4. Building a Davis Anomaly Detector
5. Custom Alerts via DQL
6. Metric Events and Settings 2.0 Schemas
7. Testing Detectors with Davis Analyzers (MCP)
8. Anomaly Volume in Your Tenant
9. Cross-Series Pointers

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Anomaly Detection app installed (custom alerts are created in **Settings** from SaaS 1.344 — § 3)
Permissions	`davis:analyzers:execute`, `settings:objects:read/write`, `storage:events:read`
MCP	Dynatrace MCP server (for AIOPS-04 / AIOPS-06 integration); analyzers also exposed in the app
Optional	Monaco / Terraform for config-as-code (see AUTOM-03 / AUTOM-04)

1. The Five Detection Mechanisms

1.1 Static threshold

Hard limit: *response time must be < 1 s*. Trips when the metric crosses the line. Configured manually. Fast to set up, but brittle — a threshold that fits Tuesday at 3 AM rarely fits Friday at noon.

Use when: an SLO, contract, or capacity ceiling defines the limit. Don't use for anything that varies with traffic.

1.2 Auto-adaptive threshold

Davis learns the metric's baseline and shifts the detection threshold as the baseline moves. No seasonal awareness — purely adaptive to recent trend.

Use when: the metric trends over time but doesn't have weekly / daily seasonality. Service throughput on a steadily-growing app is a classic fit. Newly deployed or newly changed traffic needs about a week to produce a representative baseline — see the history-window note below.

1.3 Seasonal baseline

Davis learns the metric's daily, weekly, and (where it has data) yearly seasonality. Detection considers the *expected* pattern — Tuesday at 3 AM and Friday at noon get different thresholds.

Use when: the metric has obvious recurrence — business hours, weekday/weekend differences, monthly billing cycles. Give it roughly two weeks before trusting day-of-week distinctions — see below.

1.4 Multi-dimensional (automated) baseline

Davis builds baselines per-dimension automatically — per region, per browser, per OS, per user-action. You don't configure this; the platform does it under the hood for RUM-like metrics.

Use when: you don't — Davis chooses. Your job is to let the high-cardinality dimensions through to it (don't pre-aggregate them away).

> **This is not a licence to split *your own* detectors by a high-cardinality dimension.** Davis's multi-dimensional baseline consumes the dimensions internally and still emits grouped problems. A custom detector grouped by a volatile dimension — application version, pod name, HTTP status code — emits a separate alert *per dimension value*: volume then scales with your deployment frequency, and alerts strand themselves on entities that no longer exist. Feed dimensions to Davis; do not fan your own alerting out across them. ALERT-02 §3 carries this as an anti-pattern.

1.5 Novelty detection and forecasting

Novelty flags patterns the model has never seen — sudden new error types, never-before metric shapes. **Forecasting** projects a series forward, useful for capacity planning and trend extrapolation.

Use when: you're hunting for *unknown unknowns* (novelty) or projecting growth (forecast). Both available as Davis analyzers — see Section 5.

How much history before you can trust it. Auto-adaptive detection calculates its threshold from a trailing **7-day** reference window; seasonal baseline uses a trailing **14-day** window. A service that was just deployed, or whose traffic pattern just changed, won't have a representative auto-adaptive baseline for about a week, and won't have enough history to distinguish weekday from weekend behavior for about two — a detector built on either mechanism before then is judging against an incomplete picture, not a wrong one.

> **Sources:** Auto-adaptive thresholds for anomaly detection (DT docs), Seasonal baseline (DT docs), Adjust the sensitivity of anomaly detection for services (DT docs).

2. Picking the Right Detector

A simple decision flow:

1. *Is there a hard contractual or SLO limit?* → **Static threshold.**
2. *Does the metric have recurring weekly/daily patterns?* → **Seasonal baseline.**
3. *Does it trend over time without strong seasonality?* → **Auto-adaptive threshold.**
4. *Is it RUM / high-cardinality user-facing?* → **Davis handles it; don't pre-aggregate.**
5. *Are you hunting unknown unknowns?* → **Novelty detection.**
6. *Capacity planning?* → **Forecasting.**

Anti-pattern alert: static thresholds on traffic-correlated metrics. They alert on every off-peak hour and on every traffic spike. Move to seasonal or auto-adaptive.

3. Configuration Surface: App vs. Settings vs. Code

Three places to configure detection — pick one per environment, not one per detector.

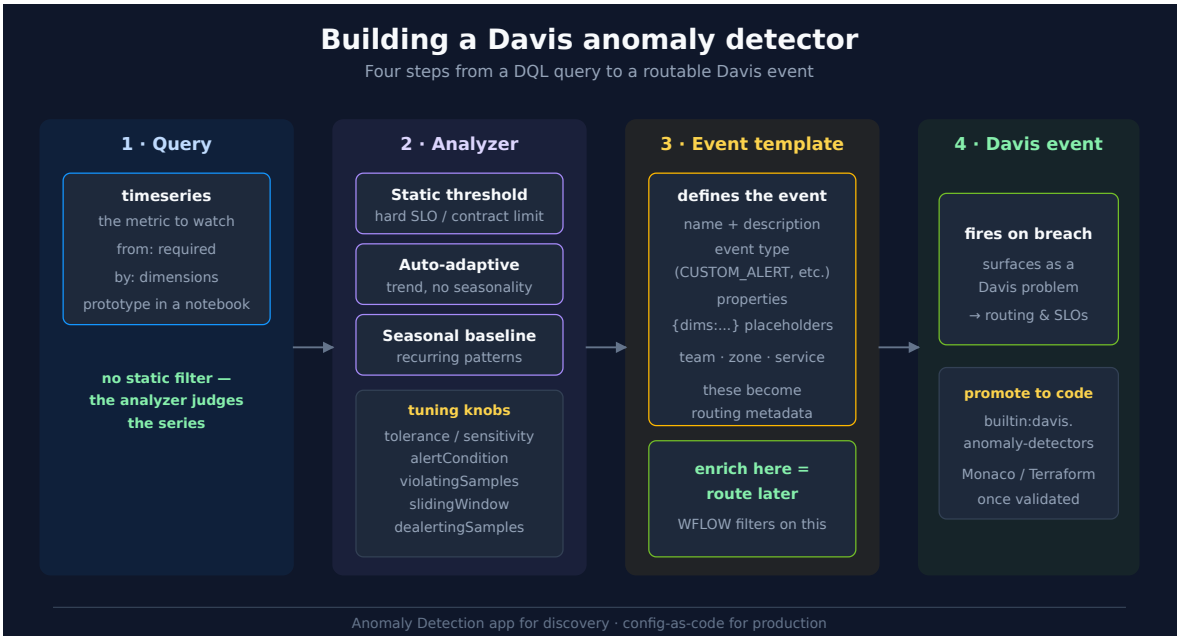
Surface	When to use
Anomaly Detection app	Exploration, one-off detectors, quick wins. Best for SREs in the moment.
Settings 2.0 (UI)	Steady-state detection that's been validated; lives under specific settings schemas.
Config-as-code (Monaco / Terraform)	Anything you want versioned, reviewable, and reproducible across environments.

> **Custom alerts move to Settings (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and the app-based build in § 4 and § 5 is the working path there.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

Production teams should converge on config-as-code. The app is the right place to *discover* what to alert on — but the moment a detector matters in production, it should live in source control. See **AUTOM-03** (Monaco) and **AUTOM-04** (Terraform) for the patterns.

4. Building a Davis Anomaly Detector

Sections 1–3 told you *which* mechanism to pick and *where* to configure it. This section walks the actual build in the **Anomaly Detection app** — the four steps that turn a DQL query into a routable Davis event. On SaaS 1.344 and later, custom alerts are created in **Settings** instead (§ 3).



Step 1 — The query contract

The detector samples a `timeseries` query on a schedule. Critically, **you do not write a `filter ... > threshold` in the detector query** — the analyzer (Step 2) is what decides whether the series is anomalous. Your query's job is to *return the metric*, grouped by the dimensions you want to alert per (`by:{...}`). A `from:` range is mandatory.

> **Develop the query in a notebook first.** This is the notebook-as-scratchpad pattern: write and run the `timeseries` here, confirm it returns the shape you expect, then paste it into the detector. A query that returns nothing in a notebook will silently never fire as a detector.

Step 1a — Records-based detectors for sparse signals

Not every signal is a metric series. For a condition that lives in raw logs or events and occurs rarely — failed logins, a specific exception, a licence error — set the detector's **Data type** to **Records** instead of a metric, and give it a deliberately large aggregation window:

```
fetch logs, from:now() - 60m
| filter matchesPhrase(content, "authentication failed")
| summarize failed_login_count = count()
| filter failed_login_count &gt; 500
```

With a 60-minute aggregation window, set the detector's **Delay** to **60 Minutes** so the evaluation window and the execution cadence line up. Two things improve at once:

- **Noise.** A sparse signal evaluated minute by minute produces false positives from ordinary clustering. Five failed logins in one minute is nothing; 500 in an hour is something. The window *is* the tuning.
- **Cost.** The detector runs once an hour rather than sixty times — a 60× reduction in evaluations against raw log data. Querying raw records on every evaluation is the expensive shape (FAQ-09, FINOPS-03), and widening the window is the cheapest mitigation short of extracting a metric at ingest (OPIPE).

> **Prototype this one especially carefully.** The query above is syntactically valid and executes, but on the tenant it was tested against `"authentication failed"` matched nothing, so `summarize` returned a single row with `failed_login_count = 0` and the final `filter` removed even that. Zero rows is precisely the silent-detector failure described above — the detector would install cleanly and never fire. Run the query *without* the threshold filter first and confirm the count is non-zero before you trust the phrase.

If the condition recurs often enough to warrant continuous watching, extract it to a metric in OpenPipeline and put an ordinary metric detector on that instead. Records-based detection is the right tool for genuinely sparse conditions, not a general substitute.

Step 2 — The analyzer and its tuning knobs

Apply one analyzer to the series:

Analyzer	Judges against	Use when
Static threshold	A fixed line	A hard SLO / contract / capacity ceiling
Auto-adaptive threshold	A learned, drifting baseline	The metric trends but has no seasonality
Seasonal baseline	A confidence band around the seasonal pattern	The metric recurs (business hours, weekly cycles)

The tuning parameters that control noise:

Parameter	What it does
`tolerance` (seasonal) / sensitivity	Width of the confidence band — higher = fewer alerts

Parameter	What it does
`alertCondition`	Direction that trips: `ABOVE`, `BELOW`, or `OUTSIDE`. `ABOVE` / `BELOW` apply to every model; **`OUTSIDE` is a baselining concept** and community guidance is that it is accepted only by auto-adaptive and seasonal models — confirm against the settings schema for your tenant before scripting it onto a static-threshold detector
`violatingSamples`	How many points in the window must violate to fire (max 60)
`slidingWindow`	How many points are evaluated together (max 60; ≥ `violatingSamples`)
`dealertingSamples`	Consecutive clean points required to close the alert (max 60)
`alertOnMissingData`	Treat absent data as a violation
query offset	Shift the evaluation window to absorb data latency

The `violatingSamples` / `slidingWindow` pair is your primary noise control — requiring, say, 3 violations out of a 5-point window suppresses single-spike flapping while still catching sustained breaches.

Step 2a — Workflow problem-trigger delay (a different layer)

SaaS 1.344 (rollout from 07/29/2026): problem triggers in Workflows can be delayed. In a workflow's Dynatrace Intelligence problem trigger you can set a delay, and *"the trigger will fire only if a problem remains open for the configured period."* This damps **notifications**, not detection — Davis still opens the problem on schedule, and every other workflow on the same problem fires on its own settings. See FAQ-21 for the Minimum duration option.

This is not an analyzer parameter, and it is deliberately not a row in the table above. The two sit at different layers, and conflating them is how tuning goes wrong:

Layer	Decides	Configured in
Analyzer parameters (`violatingSamples`, `slidingWindow`, `dealertingSamples`, tolerance)	Whether *this series* is anomalous — i.e. whether a Davis event fires at all	Per detector, in the Anomaly Detection app or `builtin:davis.anomaly-detectors`

Layer	Decides	Configured in
Workflow problem-trigger delay	Whether *this workflow* acts on a problem that has already opened — short-lived problems close before it fires	Per workflow, on the problem trigger

So the trigger delay **complements** `violatingSamples` / `slidingWindow` rather than replacing it. Its distinct value is reach: it suppresses paging for short-lived problems from detectors **you do not own** — built-in detections, and detectors configured by other teams — which no amount of analyzer tuning on your own detectors can touch. But it acts only for the workflow you set it on: the problem count does not change, and any notifying workflow without the delay keeps paging.

For any detector you build yourself, `violatingSamples` / `slidingWindow` remains the control to rely on and the right first lever. Tune the detector to stop firing spurious events; reach for the trigger delay for noise arriving from events you cannot tune at source.

Step 2b — Scoping the detector with Segments

Alongside the analyzer, the Anomaly Detection app offers **Set scope** (Simple or Advanced tab) with a **Segments** field — choose one or more segments to restrict which slice of data the detector evaluates. This lets you alert on a specific subset such as a region, cluster, or environment without rewriting the query.

> **Do not duplicate a filter the segment already applies.** Segment filters are injected into the query at execution time, so re-stating them in the `timeseries` query is redundant and makes the detector harder to reason about.

This is the only place segments touch alerting. Segments scope *detection* — which signals get evaluated — and they scope *queries*. They do **not** scope workflow triggers, notification routing, or problem visibility. If you are migrating off Management Zones and expecting segments to carry your alert routing, see MZ2POL-09: routing moves to problem-triggered workflows filtered on affected-entity tags, not to segments.

Step 3 — The event template (this is what makes routing possible)

When the detector fires it emits a Davis event whose shape *you* define: an event **name** and **description** (both support `{dims:...}` placeholders that interpolate the breaching

dimensions), an event **type** (`CUSTOM_ALERT` , `ERROR_EVENT` , `PERFORMANCE_EVENT` , …), and **properties** — key/value pairs such as team, zone, or service.

Those properties are the metadata a downstream workflow filters on. **Enrich here or you cannot route later** — a detector that fires a bare event with no team/zone property forces every workflow to re-derive ownership from the affected entity. Spend the effort in the template.

Attribute the event to a real entity, or it correlates against the wrong one. Alongside name, description, type and properties, a Davis event carries `dt.smartscape_source.id` — the Smartscape entity ID of whatever the signal is *about*. Davis's universal correlation rule merges every event naming the same entity into a single problem (AIOPS-03 §1). Set it to an actual host, service, or workload ID, normally interpolated from a `by:{}` dimension the query already groups on.

The documentation asks for this field to be set "to an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string" — it does not say where a detector's event lands when the field is left unset. The one documented fallback belongs to the Events API v2: an event posted without an `entitySelector` "is associated with the environment" entity. For detector events no page says where an unset source lands. On a validation tenant (7 days to 10/12/2026), most problem-eligible events without one were still attached to a real entity, but those attached to the environment entity alone collected in large problems: 14.7 events per problem against 1.1 for the rest. Either way no threshold change fixes it; only the event template does (AIOPS-03 §1). Section 8 has a query that finds these in your own tenant.

Two event types that never open a problem on their own. `CUSTOM_INFO` (severity 5, Informational) and `WARNING` (severity 4, Warning) are stored in Grail, are fully queryable, and can trigger workflows, but they do not raise problems; correlation can still group them into a problem that something else opened. That makes them useful for chronic issues already tracked elsewhere, for routing an observation to Slack or Jira without cluttering the Problems app, and — most valuably — for calibration:

> **Shadow-deploy a new detector.** Rather than guessing a threshold, ship the detector with `event.type` set to `CUSTOM_INFO` , leave it running for two to four weeks, then measure how often it *would* have fired (Section 8). Tune against that evidence, and only then promote it to `CUSTOM_ALERT` . This turns threshold-setting from a guess into a measurement, and being wrong during the observation window costs nothing but storage.

Step 4 — Fire, then promote

On breach the event surfaces as a Davis problem and flows into routing (WFLOW series) and SLO error budgets. Once a detector matters in production, move it out of the app and into config-as-code (`builtin:davis.anomaly-detectors` — see Section 6) so it is versioned and reviewable.

Step 5 — Pre-Publish Checklist

Three checks, run before a detector leaves the notebook-as-scratchpad stage — each one closes a failure mode this section already describes, so this is packaging, not new material:

- **Cardinality.** Does the `by:{}` grouping include a volatile dimension — pod name, application version, HTTP status code? Section 1.4's anti-pattern applies directly: a detector grouped by a high-cardinality field opens one alert per value, and volume then tracks deployment frequency rather than incident rate. Count the distinct values the grouping would produce before publishing, not after:

```
dql // Cardinality check — how many distinct alert identities would this grouping produce? // Swap in whatever by:{} dimension the detector will use. timeseries avg(dt.kubernetes.container.cpu_usage), by:{dt.entity.cloud_application_instance}, from:-7d | summarize distinct_entities = countDistinct(dt.entity.cloud_application_instance)
```

- **Attribution.** Does the event template set `dt.smartscape_source.id` to an existing entity ID, as the docs recommend (Step 3)? Events attached to the environment entity alone end up in large problems with no usable root cause (Section 8).
- **Cost.** Was the query prototyped in a notebook first (Step 1), and — for a records-based detector — is the aggregation window wide enough to avoid a per-minute scan of raw data (Step 1a)? A detector that scans expensively on every evaluation compounds the cost of every firing, noisy or not.

None of these three are analyzer tuning — they're structural, and a mistuned analyzer sitting on a structurally broken detector still fires wrong.

> **Sources:**
>
> - Anomaly detection configuration (DT docs)
> - Set up anomaly detectors via API (DT docs)
> - Avoid overalerting (DT docs) — *"make sure to set the dt.smartscape_source.id field to an existing Smartscape entity ID, like a host or service entity ID rather than an arbitrary string"*
> - Ingest an event — POST /api/v2/events/ingest (DT docs) — *"If not set, the event is associated with the*

> *environment"* (scoped to events posted through this API)
>
> - What's new in Dynatrace SaaS 1.344 (DT docs)
> - Standardized event severity (DT docs) — *"they don't raise problems on their own"* (Warning and Informational events)

5. Custom Alerts via DQL

Beyond pre-canned detectors, Anomaly Detection app supports **DQL-based custom alerts** (created in **Settings** from SaaS 1.344 — § 3). You write the query, the app evaluates it on a schedule, and a breach generates a Davis event.

Example — alert when a service's error rate breaches 5% for 5 minutes. Following the Step 1 contract in §4, the detector returns the error-rate series and the static-threshold analyzer applies the 5% line:

```
// Detector query: return the error-rate SERIES per service - no threshold
filter.
// Configure the threshold in the analyzer: Static threshold 5, alertCondition
ABOVE,
// violatingSamples 5 / slidingWindow 5 = "above 5% for 5 minutes".
timeseries {
    failures = sum(dt.service.request.failure_count, default:0),
    total    = sum(dt.service.request.count)
  },
  by:{dt.smartscape.service},
  from:-1h, interval:1m
| fieldsAdd error_rate = (failures[] / total[]) * 100
| fieldsKeep timeframe, interval, dt.smartscape.service, error_rate
```

Watchpoints when writing custom-alert DQL:

- Always include a `from:` time range. Detectors that omit it are rejected.
- Use named-parameter `decimals:`, `then:`, `else:` where required (`round`, `if`, `substring`).
- Aggregations used downstream need explicit aliases (`error_rate = ...`, `mttr = ...`).
- Element-wise array operations on timeseries: `failures[] / total[]` returns an array. Keep the series as an array for a detector — `arrayAvg` + `filter` is an ad-hoc notebook check, not a detector query.
- Use `default:0` on sparse counters so zero-failure intervals count as 0, not null. Without it, an average over the array covers only the minutes that had failures and overstates the rate.

6. Metric Events and Settings 2.0 Schemas

Not every alert needs the DQL-based detector. **Metric events** are the classic Settings 2.0 mechanism for threshold / baseline alerting directly on a metric key — lighter weight than a DQL detector when you are alerting on a single pre-existing metric.

Mechanism	Settings 2.0 schema	Upgrade status	Reach for it when
DQL-based Davis anomaly detector	`builtin:davis.anomaly-detectors`	Carries forward	The signal needs a query — joins, derived fields, multi-metric logic. Also the migration target for metric events
Metric event	`builtin:anomaly-detection.metric-events`	**Removed in Latest Dynatrace**	Only where you already have one; do not author new ones

Prefer the Davis anomaly detector for anything new. `builtin:anomaly-detection.metric-events` is on Dynatrace's list of Settings 2.0 schemas removed in Latest Dynatrace, and none of the schemas on that list are visible there — so once the tenant moves to the latest Dynatrace, every metric event has to be recreated as a DQL-based detector, and any configuration-as-code that references the schema by ID has to be updated first. (On a tenant observed 07/31/2026, the ready-made *Check your upgrade readiness* dashboard flagged the same schema **Blocked at upgrade**.) Dynatrace publishes a transformation path, with the important limitation that **only metric *selectors* can be transformed**: metric *key* events, which put a static threshold on a single metric, have no automated conversion. The transformer also cannot tell whether the metric's data or its tags actually exist in the target, so every converted detector needs validating by hand afterwards.

That reverses the older advice in this section. A metric event is still the lighter-weight mechanism, and one you already own is fine to leave running for now — but it is a construct with an expiry, so new work should not be authored against it, and a static threshold ported as-is is in any case the main source of Gen3 alert noise (ALERT-02).

The Davis detector remains a first-class config-as-code target: provision it with Monaco or Terraform exactly as in **AUTOM-03 (Monaco) / AUTOM-04 (Terraform)**, with the schema name as the `--settings-schema` / resource selector. Check any schema's status in AUTOM-02's catalog before building a long-lived project around it.

> **Sources:**
>
> - Metric events (DT docs)
> - Anomaly detection configuration (DT docs)
> - Upgrade metric alerting (DT docs) — *"Only metric selectors are available to transform."* and *"the check can't detect whether the data from the metric is available or the necessary tags are present"*
> - Settings 2.0 schemas removed in Latest Dynatrace (DT docs) — lists `builtin:anomaly-detection.metric-events`; *"None of the schemas on this page are visible in Latest Dynatrace."*

7. Testing Detectors with Davis Analyzers (MCP)

The Dynatrace MCP server exposes Davis analyzers as callable tools. Useful when you want to test a detector against historical data before wiring it to a setting or a workflow.

MCP tool	What it does	Permission
`mcp__dynatrace__static-threshold-analyzer`	Test a static threshold against a series	`davis:analyzers:execute`
`mcp__dynatrace__seasonal-baseline-anomaly-detector`	Detect anomalies vs. seasonal pattern	`davis:analyzers:execute`
`mcp__dynatrace__adaptive-anomaly-detector`	Detect anomalies vs. learned baseline	`davis:analyzers:execute`
`mcp__dynatrace__timeseries-novelty-detection`	Flag never-seen patterns	`davis:analyzers:execute`
`mcp__dynatrace__timeseries-forecast`	Forecast future series; capacity planning	`davis:analyzers:execute`

Workflow pattern: write a `timeseries` query that returns the metric you care about, then pass the query to the analyzer. The analyzer's output is structured (anomalies, forecast bands, novelty flags) and can be embedded in workflows or notebooks.

All five analyzers also have GUI surfaces in the Anomaly Detection app — the MCP tools are the same engine accessed programmatically.

8. Anomaly Volume in Your Tenant

Davis events are the raw signals before Causal AI groups them into problems. Look at the category breakdown to understand what kinds of anomalies your detectors are firing.

```
// Davis event volume by category — last 24h
fetch dt.davis.events, from:-24h
| summarize count = count(), by:{event.category}
| sort count desc
```

```
// Active custom-alert problems (DQL-based custom alerts surface here)
fetch dt.davis.problems, from:-7d
| filter event.category == "CUSTOM_ALERT"
| summarize count = count(), by:{event.status}
| sort count desc
```

Reading the result: A high `CUSTOM_ALERT` count means your custom-DQL detectors are firing — review whether they're noisy. A high `RESOURCE_CONTENTION` or `ERROR` count means out-of-the-box auto-adaptive / seasonal detection is doing real work.

Finding your noisiest detectors

The category breakdown tells you what *kinds* of anomaly fire. This tells you which specific detectors, and — the part that matters most — whether each one is attributable and navigable:

```
// Noisiest non-informational detectors over 7 days.
// The two ID columns answer different questions - see the notes below.
fetch dt.davis.events, from:-7d
| filter not in(event.category, {"INFO", "WARNING"})
| summarize alerts = count(), by:{event.name, dt.settings.object_id,
dt.smartscape_source.id}
| sort alerts desc
| limit 20
```

Real output from a demonstration tenant over seven days, **measured 07/30/2026**, abbreviated to the top rows. Treat the absolute counts as a snapshot, not a constant — they move with what the tenant is doing; the *shape* (a high-volume detector with a null `dt.smartscape_source.id`) is the durable part:

`event.name`	`alerts`	`dt.settings.object_id`	`dt.smartscape_source.id`
`YOU ARE A PROBLEM`	5,378	*null*	*null*
`Response time degradation`	2,439	*null*	`SERVICE-6C36694E683AD694`
`Failure rate increase`	1,022	*null*	*null*
`DYNATRACE USER LOGIN`	285	`builtin:davis.anomaly-detectors` …	*null*
`Backoff event`	254	`builtin:anomaly-detection.kubernetes.workload` …	`K8S_DEPLOYMENT-79081F0B98463CC2`

Read the top row first. A custom detector firing 5,378 times in seven days with **no `dt.smartscape_source.id`** is the anti-pattern from Section 4 in its natural habitat. What it is *not* is one problem per firing — that reading is wrong, and it sends you hunting the wrong symptom. Re-measured on the same tenant over 7 days on 08/11/2026, this detector's firings collapsed to roughly **11 firings per correlation**. The damage is unrelated alerts sharing a problem and no usable root cause — not problem-count inflation. Re-tuning the threshold would not help either way. The event template has to name a real entity.

Then read the two ID columns, which answer different questions:

Column	Question it answers	Null means
`dt.smartscape_source.id`	Is this alert attributed to the entity it is about?	The template did not name the entity it is about. Check `affected_entity_ids`: if it holds only the environment entity, the alert will be grouped with unrelated ones
`dt.settings.object_id`	Can I navigate to the configuration behind it?	No settings object; identify it by `event.name` instead

`dt.settings.object_id` identifies the configuration that produced the event, so a populated value leads straight to the detector's configuration. A useful property, observed in the values returned on 07/30/2026 rather than a documented contract: it encodes both the settings

schema and the **scope**. The values abbreviated above resolve to `builtin:davis.anomaly-detectors` scoped to the tenant, and `builtin:anomaly-detection.kubernetes.workload` scoped to one specific `KUBERNETES_CLUSTER`. That scoping is why a single event name legitimately appears under several distinct object IDs — one per cluster — rather than indicating duplicated configuration.

> **Two caveats.** `dt.settings.object_id` is marked **experimental** in the semantic dictionary (`dt.smartscape_source.id` is `stable`) — fine for triage, not something to hard-code an integration against. And a seven-day unfiltered scan of `dt.davis.events` is not cheap: the run above scanned 12.9 GB and the 30-day version below 52.6 GB on 07/30/2026; the same 7-day scan read about 5.9 GB on 09/24/2026 — scan cost moves with event volume. Narrow the window for routine checks (FAQ-09, FINOPS-03).

For what counts as "too noisy" in the first place, ALERT-99 §3 carries the 0.1%-of-observed-time yardstick and the audit cadence that uses it.

Measuring what a shadow-deployed detector would have done

The calibration pattern in Section 4 needs evidence before promotion. This measures the firing frequency of a detector running in observation mode as `CUSTOM_INFO` or `WARNING`:

```
// How often would this observation-mode detector have fired?
// Replace the event.name with your own detector's name before running.
fetch dt.davis.events, from:-30d
| filter in(event.category, {"INFO", "WARNING"})
| filter event.name == "NR ALERT: Host CPU &gt; 80% sustained 5m"
| summarize firing_count = count(), by:{day = bin(timestamp, 24h)}
| sort day desc
| limit 10
```

Measured 07/30/2026: run against a ported static threshold held in observation mode, the ten most recent daily counts came back **236, 347, 459, 504, 470, 474, 502, 490, 480, 470** — the first of those a still-running partial day.

That detector must not be promoted. Roughly 470 firings a day would mean up to roughly 470 problems a day from a single rule — correlation would merge some of them, but nothing like enough — against a healthy-detector expectation of one or two firings a *month* (ALERT-99 §3). The evidence says retune before promoting: widen the sliding window, raise the threshold, or — since this is CPU on hosts, a traffic-correlated signal — move it off a static threshold onto auto-adaptive entirely (Section 1).

That is the whole value of the shadow deploy. Promoted straight to `CUSTOM_ALERT`, the same rule would have paged someone several hundred times a day before anyone discovered the threshold was wrong; held at `CUSTOM_INFO`, it cost nothing to learn the same thing.

> **Sources:** Avoid overalerting (DT docs), Anomaly detection configuration (DT docs). **Derived:** the promote/retune verdict applies the page's 0.1%-of-observed-time rule to the daily counts measured 07/30/2026.

9. Cross-Series Pointers

- **AUTOM-03, AUTOM-04** — anomaly detection settings as code (Monaco, Terraform)
- **WFLOW** — once a detector fires, route the resulting Davis event into a notification or remediation workflow
- **DBMON, CLOUD, K8S** — domain-specific anomaly patterns sit in those series; this notebook is the canonical reference for the *mechanisms*
- **AIOPS-05** — the AI models behind these detectors (causal correlation, predictive baseline, seasonal)
- **AIOPS-06** — agentic patterns: scheduled analyzer runs in workflows

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.